

Usage & Engagement Analytics Dashboard (Admin)

SRS DOCUMENT 1 — Usage & Engagement Analytics Dashboard (Admin)

1. Executive Summary

The Usage & Engagement Analytics Dashboard is an internal admin module for RhoFi.

Its purpose is to track, measure, and visualize how users interact with various features of the platform.

This helps in product optimization, identifying user patterns, and improving decision-making.

2. Business Problem

RhoFi currently lacks visibility into how users navigate the platform, what features they use, and where they drop off.

Without analytics, product improvements rely on assumptions instead of real data.

3. Module Objectives

- ✓ Track user interactions (events)

- ✓ Summarize engagement trends
 - ✓ Provide actionable insights to product leadership
 - ✓ Allow export for external analysis
 - ✓ Build a scalable analytics framework for future ML personalization
-

4. User Personas

Persona	Role	Needs
Founder (Sekar)	Business Owner	Must see what features users engage with
Product Lead (Rajani)	Business Strategy	Insights for product roadmap
Tech Lead (Boopalan)	Backend	Event consistency, API quality
Intern Developers	Implementation	Clear requirements for execution

5. Functional Requirements

5.1 Event Tracking

System must log events like:

- Page view
- Button click
- Search
- Watchlist add/remove
- Portfolio simulation
- MF comparison
- RSA query

5.2 Event Data Format

Each event must store:

- timestamp
- user_id
- session_id
- event_type
- module_name
- metadata (JSON)

5.3 Analytics Dashboards

Admin must be able to view:

Daily / Weekly / Monthly Active Users

Top 10 Features Used

Average Session Duration

Heatmap: Most Active Time of Day

Funnels (Optional Phase-2)

Drop-off points

5.4 Filters

Dashboard must support:

- Date range
- User type
- Feature category

5.5 Export

Admin must export CSV containing:

- Event details
 - Summary metrics
-

6. Non-Functional Requirements

Category	Requirement
Performance	API response < 300ms
Scaling	Should support 10M events
Security	Admin-only access
Logging	Error logs + access logs
Observability	Event counts via dashboards (later)
Consistency	Event schema must remain compatible

7. Data Model

Table: event_log

Column	Type	Description
event_id	UUID	Primary Key
user_id	UUID	FK
session_id	UUID	User session
event_type	String	page_view, click, etc.
module	String	“RSA”, “MF”, “Watchlist”, etc.
metadata	JSON	Dynamic fields
created_at	Timestamp	Time of event

8. API Specifications

POST /event/log

Request Body:

```
{  
  "user_id": "UUID",  
  "session_id": "UUID",
```

```
"event_type": "click",
"module": "RSA",
"metadata": { "button": "scan" }
}
```

GET /analytics/summary

Response:

```
{
  "daily_active_users": 120,
  "top_features": [...],
  "session_duration_avg": 148
}
```

GET /analytics/export

Returns CSV.

9. User Flow

1. User performs an action
 2. Event sent to backend
 3. Event stored in DB
 4. Aggregation API calculates analytics
 5. Admin views dashboard
-

10. UI Wireframes (Low Fidelity)

- Sidebar → Usage Analytics
- Widgets grid:
 - DAU chart
 - Feature ranking
 - Time-of-day heatmap
 - Session duration

(Gopal will generate wireframes separately on **REQUEST**.)

11. Dependencies

Area	Person
Backend help	Shafvan / Boopalan
Infra	Boopalan
UI Design	Gopal
Business clarity	Sekar & Rajani

12. Risks & Mitigation

Risk	Mitigation
High volume events	Batch insert later
UI complexity	Start with simple charts
Mislogged events	Setup event validator

13. Acceptance Criteria

- ✓ Event tracking fully functional
 - ✓ Dashboard with at least 6 widgets
 - ✓ Filters working
 - ✓ Export working
 - ✓ APIs tested
 - ✓ Code reviewed & merged
-

14. 1-Month Plan & KPIs

- **Week 1: Understanding + event logging**
- **Week 2: APIs + data model + admin UI skeleton**
- **Week 3: Charts, filters, export**
- **Week 4: QA, polishing, final demo**

KPIs:

- 90% of required widgets built
 - <300ms API responses
 - 100% filter coverage
 - 1 demo delivered to Founder
-